

UAS Contextual Authentication Developer Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS Contextual Authentication Developer Guide.....	3
2. Introduction to UAS Contextual Authentication for Developers.....	4
3. UAS Contextual Authentication Implementation Using API.....	8
4. Appendices.....	13

1. Preface - UAS Contextual Authentication Developer Guide

AccessMatrix UAS (Universal Authentication Server) provides contextual authentication components that authenticate the users based on their contexts, like users' geolocation, login time, device status (whether it is rooted device or new device) or based on the remote address.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Intended Reader

This guide is intended for experienced software developers who are familiar with the Authentication API.

Document Scope

This document's scope explains the concept of contextual authentication and guides you to the application integration using AccessMatrix Contextual Authentication solution.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [UAS Administration Guide](#)
- [AccessMatrix REST API Specification](#)

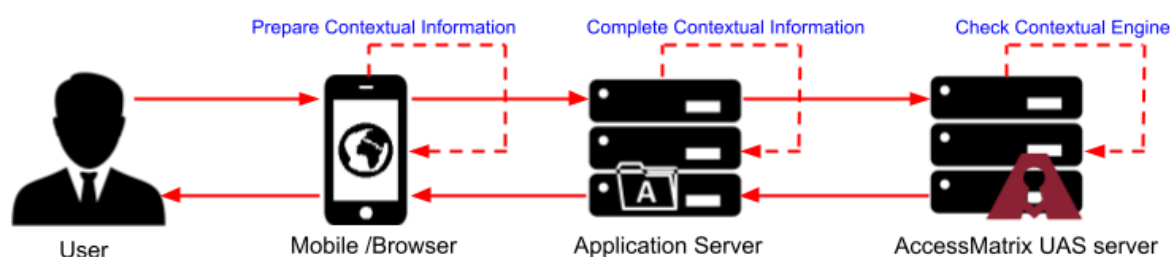
2. Introduction to UAS Contextual Authentication for Developers

This document shows you how you can carry out the contextual authentication solution in AccessMatrix UAS. After reading this document, readers are expected to:

- Understand the concept of UAS Contextual Authentication.
- Understand the use case and workflow of UAS Contextual Authentication.
- Be able to implement UAS Contextual Authentication by integrating related REST APIs.

Contextual Authentication Concept

In general, contextual authentication is about an authentication process where the contexts (e.g. location, time, etc.) of the authenticating users are taken into account to decide whether the users are denied or granted access or prompted for another authentication method.



In AccessMatrix UAS, contextual authentication is a server-side component of UAS that provides non-static authentication, which focuses on providing dynamic security to enhance usability for users and strengthen security based on configured policies.

The users' contexts are used to determine the users' profiles. The contexts are users' geolocation, historical geolocation, login time, IP addresses and devices. During the authentication process, UAS will consider the contexts of the authenticating user to the system to determine the risk profile associated with the login. Therefore, the backend application must provide contextual information and pass it to the AccessMatrix UAS server via login API.

UAS then uses the user's risk profile to determine the complexity of the authentication challenge. Higher risk profiles lead to stronger challenges. This risk-based implementation allows the application to challenge the users for additional credentials only when appropriate risk level.

Before starting the integration, you need to create and configure a contextual policy in AccessMatrix Admin Console and set it to the authentication realm you want to use for your application. UAS only supports contextual authentication via its login API. Note that Login API needs an authentication realm that has a contextual authentication policy being set.

AccessMatrix provides three types of contextual authentication policies:

- **Score-Based Contextual Authentication Policy**, which uses the users' contexts to calculate a risk score.
- **Rule-Based Contextual Authentication Policy**, which allows for the configuration of follow-up actions that are each based on a single contextual parameter.
- **Script-Based Contextual Authentication Policy**, which allows for the configuration of follow-up actions that are each based on one or more contextual parameters.

From an application integration perspective, there is no difference between both authentication policies. The API you use will be the same (login API), and the contextual parameters you send will depend on the business requirements. For example, the business might mandate that if the user logs in from certain countries and/or uses a new device, the application must prompt for 2FA.

Score-Based Contextual Authentication

The score-based contextual authentication assesses each authentication's risk score during the login process. It will then use the score to determine relevant action (i.e. deny, allow or escalate to challenge stronger authentication factor). In addition, the application is responsible for supplying information such as country, time, historical geolocation, whether the device is new or rooted or accessed from a remote address.

For example, starting with an initial risk score of 0, if a user logs in from certain countries, a risk score of 40 will be added. If the user login out of office hours, then a risk score of 30 will be added to the total risk. If the user uses a new device, then it will add a risk score of 50. Once the risk score crosses a certain defined threshold, certain actions can be taken, such as denying log in or requiring a stronger authentication such as 2FA. If the total risk score is more than 100, then deny access to the user; else, if the risk score is between 30 and 100, then prompt for 2FA, otherwise grant access to the user.

Rule-Based Contextual Authentication

Rule-Based Contextual Authentication assesses if a single specified contextual parameter matches the configured policy and then takes action immediately if so. For example, if the user logs in from certain countries, then deny access. If the user uses a new device, then prompt for 2FA. If the user uses a rooted device, then deny access.

The Rule-Based Contextual Authentication policy also supports notification-only policies. So, instead of defining whether to grant or deny access or prompt for 2FA based on the rule, the application can set the action to only provide a notification if a user (for example) uses a new device.

Script-Based Contextual Authentication

Script-based Contextual Authentication allows you to use scripted expressions to create conditions for an action to be taken. This enables the creation of rules that are conditional on multiple contextual parameters. Do note that this differs from score-based contextual authentication - the specified contextual parameters are treated as conditions for an action instead of being weighed in terms of a risk score.

Rules can be created using the NOT, AND, and OR logical operators. For example:

- **NOT** - If a user does not log in from the specified countries, prompt for 2FA. Otherwise, deny access.
- **AND** - If a user logs in within working hours and logs in from the specified country, prompt for 2FA. If only one or neither of these conditions are met, deny access.
- **OR** - If a user logs in within working hours or logs in from the specified country, prompt for 2FA. If both are true, prompt for 2FA as well. Only if neither of these conditions are met is access denied.

Notification-only policies are also supported.

Which Contextual Policy to Use?

Which contextual policy to use depends on the business requirements.

If you want to consider all of the relevant contextual information before deciding on the actions, choose Score-Based Contextual Authentication Policy. For example, an enterprise mandates the application to

consider three context information during login, i.e. origin country, time and device. Each context information is assigned with a risk score. The actions will depend on the total risk of those three contexts; for example, a login attempt with a total risk score of more than 100 will be denied.

If you want to treat each context separately and take action based on the individual contexts, you may choose Rule-Based Contextual Authentication Policy. For example, if the enterprise mandates to deny login if the user uses the rooted device, then you choose a rule "If user's device is rooted and set the action to "deny login". If the enterprise mandates to prompt 2FA if the user logs in from certain countries, you choose a rule "If within this country list" and set the action to "1FA + 2FA". Alternatively, if the enterprise wants to notify the user via email or SMS when they log in from a new device, then choose "If the device is new", set the action to "Send Notification" and set the event notification policy (refer to [AccessMatrix Common Administration Guide](#) to configure an event notification policy).

Lastly, if you require a more sophisticated rule-based policy where an action is conditional on multiple contextual parameters, choose Script-Based Contextual Authentication Policy.

For more information on setting up score-based or rule-based contextual authentication policy, refer to [UAS Administration Guide - Administration for Contextual Authentication](#) .

3. UAS Contextual Authentication Implementation Using API

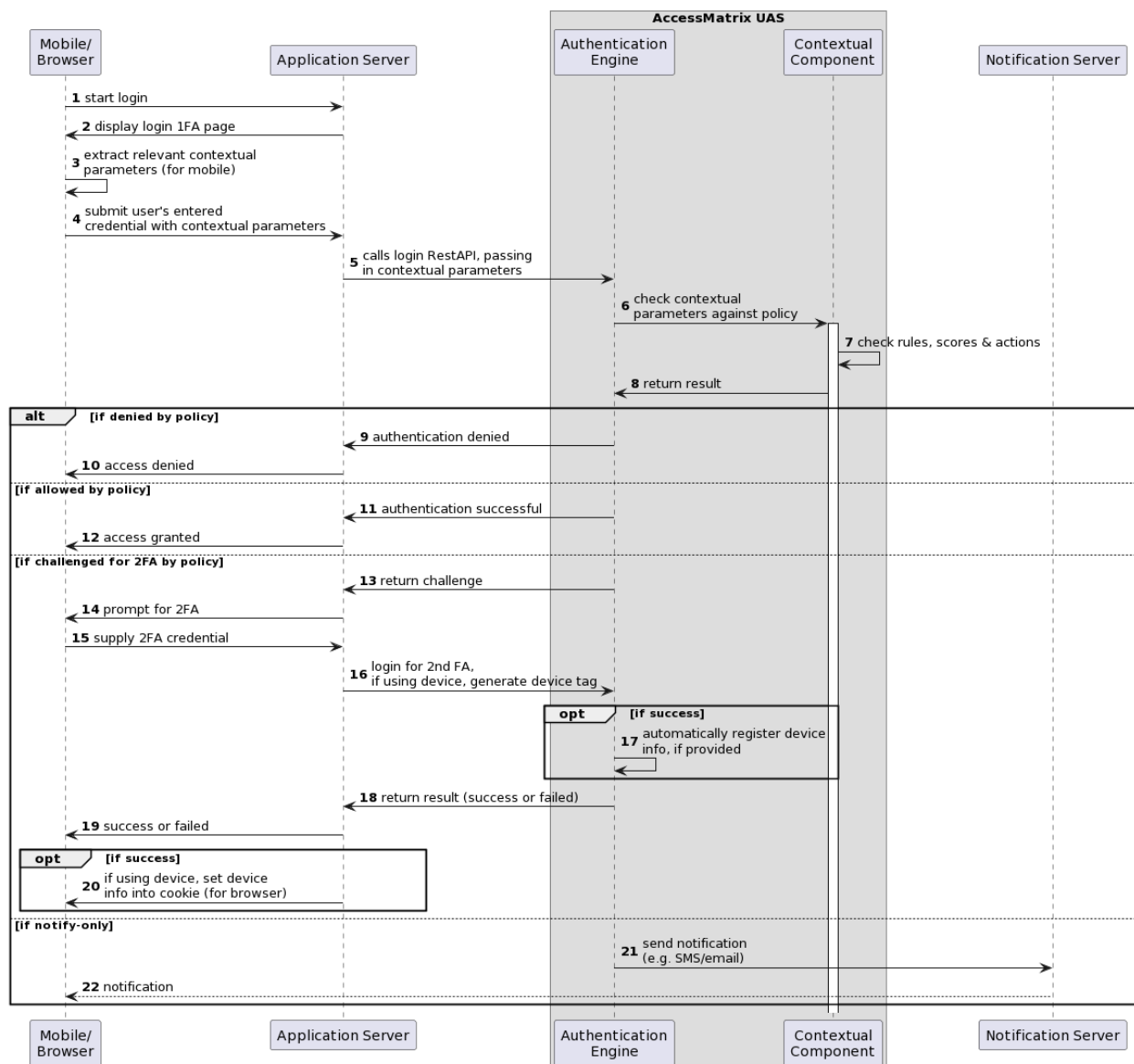
API Use Case and Sample Codes

UAS contextual authentication is only supported by Login API (part of Authentication Service). For a complete reference of Login API, refer to the [AccessMatrix REST API Specification](#).

For this login use case to work successfully. In the AccessMatrix UAS module, you need to have:

1. Authentication realm (with login modules) created and assigned to the user.
2. Contextual authentication policy configured to conform to an organization's compliance standard and then assign it to the realm.
3. If you want to use a country as part of the contextual authentication policy, you must obtain and set up the geo-location database in AccessMatrix for country resolution. To find out more, refer to [UAS Administration Guide](#) for the configuration and setup details.

Sequence Diagram



As can be seen from the sequence diagram above, in step #2, the application server must prepare all of the contextual parameters that will be sent to UAS.

Contextual Authentication Parameters

The application must provide contextual information like location, remote IP, and device info to the UAS for contextual evaluation.

Policy Parameter	Login API Parameter	Description
High-Risk Countries or Geolocation History	(under "params") User's device IP address or "context.latitude" / "context.longitude"	Country can be used to define contextual policy. For example, if a user logs in from certain countries, add a risk score or deny login. Or, if a user logs in from a new country, prompt 2FA. The country is looked up from a geo-location database. You can choose to use an IP-based or latitude/longitude-based geolocation database (refer to UAS Administration Guide). Latitude/longitude should be sent from the user's device/browser-based on GPS detection.
Time-of-day	Login time (server time)	Server time will be used in the policy.
Device Rooted Status	(under "params") "context.isRooted"	This parameter shall be prepared from the mobile app, where it checks the phone whether the phone is rooted/jailbroken or not.
New Device	"device": { "type": "iOS", "id": "string", "tag": "string", "agent": "string", "desc": "string" }	This parameter is a JSON object containing the device type, id, tag, agent and description. Refer to AccessMatrix REST API Specification documentation for a complete explanation. "Type", "id", and "tag" are mandatory. Thus if the device is set without setting those three mandatory items, UAS will throw an exception. If either "type", "id", and "tag" do not match, or if the overall "device" object is not set, then the contextual authentication component will treat them as a new device.
Remote Address	User's device IP address	You can configure the policy to check if the user's IP address is within a certain range. For example, if a user logs in from a certain IP range, increase the risk score (Score-Based Contextual Policy) or perform 2FA (Rule-based Contextual Policy). Refer to the UAS Administration Guide for information how to configure it.
User Agent	(under "device") "agent" : "string",	The User-Agent is a standard HTTP header attribute that identifies the characteristics of the client-side application, such as the user's web browser, operating system, and device. You can configure the contextual authentication policy to check if the User-Agent string of the user's client-side application contains a match with the regular expression value specified in the policy. For example, you may wish deny login for a user logging in from a specific web browser based on the User-Agent string.

Policy Parameter	Login API Parameter	Description
Custom Contextual Authentication Parameter	"context.vars":{ "variable" : "value" }	AccessMatrix UAS supports user-defined contextual authentication parameters for the Script-based Contextual Authentication Policy only. This parameter is a JSON object containing custom contextual authentication variables defined by the user. Multiple variables can be added and different data types can be used, depending on the user's needs. Nested JSON objects and arrays are also supported. Refer to UAS Administration Guide - Adding Custom Rules for Script-Based Contextual Policy for information how to configure custom rules in the Admin Console.

REST Sample Codes

Contextual authentication is only supported by Login API.

Login API Call	
URL: /authn/login	
Request	
JSON	<pre>{ "sessionToken": "1000115sNGcPgF808...y5FrdB5f009", "userId" : "UASUser", "password" : "password.1234", "device" : { "type" : "Browser", "id" : "WebDevice_001", "tag" : "Tag_001", "agent" : "Mozilla Firefox", "desc" : "User Browser" }, "params": { "realmId" : "LoginRealm", "remoteAddr" : "190.0.0.1", "context.latitude": "1.32565", "context.longitude": "103.92147", "context.isRooted": "false", "context.malwareDetected" : "false", "context.botOrAutoTrxDetected" : "false" "..." "context.vars":{ "transactionType":"MoneyTransfer", "transactionAmount":10000 } } }</pre>

Login API Call

```
}  
}
```

Response

JSON

```
{  
  "result": {  
    "authenticated": true,  
    "sessionIID" : "EEPCmk3zSnj2tYT_7mTBow",  
    "sessionToken" :  
    "10001oft01b1HNEV9StsxsemrRmpdkXHyjvrp8bq...-  
    ItWjcpGU0pNzT9UO9tkFFuBE",  
    "userUUID": "71SwCGOoB-",  
    "loginLocation": "190.0.0.1",  
    "loginModuleStates": [  
      {  
        "authenticated": true,  
        "..."  
      }  
    ],  
    "remoteAddr": "190.0.0.1"  
    "..."  
  }  
}
```



Note: Refer to the [AccessMatrix REST API Specification](#) documentation for more information about the authn/login API, such as responses, exceptions, error codes.

4. Appendices

A. Sample IP Address

For testing, you need to simulate login from various countries. Here is a list for your convenience.

You can either hard code these IP addresses or configure your system to these IP addresses:

- 119.74.224.0 Singapore, Bedok New Town
- 119.81.94.8 United States, Wisconsin, Institute
- 119.40.96.0 Mongolia
- 119.18.192.0 China, Beijing



Note: IP address geolocation changes all the time. If you update the geolocation database file, then these countries might change.
